Lab#4



– Digital Electronics –

Building and programming a simple RISC-V system

The purpose of this laboratory session is to learn how to build and program a simple RISC-V-based system

Contents:

1. Building a NIOS V (RISC-V) system
2. Do it yourself

Abbreviations and acronyms:

IC – Integrated Circuit
LED – Light Emitting Diode
MUX – Multiplexer

USE EXACTLY THE SAME I/O PINS SPECIFIED IN THIS DOCUMENT.

1 – Building a NIOS V system

Follow the tutorial in the appendix. It explains you how to build a simple NIOS V system with QSYS (Platform Designer) and how to run a simple program.
NOTE: this document is based on Quartus Prime 24.1std Lite Edition and on Intel-Altera application note AN 985: Nios V Processor Tutorial.

PLEASE REFER TO THE LAB#0 FILE FOR DETAILS ON PROGRAMMING THE FPGA

2 – Do it yourself

a. **Create** a new Quartus project (*lab_nios_system*) with the following ports which correspond to names already defined in the pin assignment .csv file.
   - CLOCK_50: in std_logic / input CLOCK_50
   - KEY: in std_logic_vector(3 downto 0) / input [3:0] KEY
   - SW: in std_logic_vector(7 downto 0) / input [7:0] SW
   - HEX4: out std_logic_vector(0 to 6) / output [0:6] HEX4
   - HEX5: out std_logic_vector(0 to 6) / output [0:6] HEX5
   - LEDR : out std_logic_vector(7 downto 0) / output [7:0] LEDR

b. Stemming from the tutorial **create** with the QSYS (Platform Designer) Tool a NIOS V processor system (*lab_nios_processor*) with
   - one NIOS V/m processor
   - at least 256kiB on-chip memory (increase if required)
   - one 2 bit input parallel port
   - one 8 bit input parallel port
   - one 8 bit output parallel port
   - two 7 bit output parallel port
   - a JTAG UART

c. **Declare** *lab_nios_processor* as a component inside *lab_nios_system* and **instantiate** it connecting:
   - the processor clock to CLOCK_50 and the reset to KEY(0).
   - the 2 bit input port to KEY(2), KEY(3), and the 8 bit input port to SW
   - the 8 bit output port to LEDR and the two 7 bit output ports to 7 segment displays HEX4 and HEX5 respectively.

d. **Compile** the hardware system and verify correctness. Then download it on the DE1-SoC board.

e. **Write, compile and execute** the 3 different programs doing the following tasks:
   - Connect SW to LEDR
   - Encode SW(7) SW(6) SW(5) SW(4) into 7 segment display HEX4. The mapping from the binary input to the output is the one that makes input 0000 corresponding to ‘0’ on the display, 0001 corresponding to ‘1’ and so on till 1111 corresponding to ‘F’.
   - Count the rising transitions of the signal coming from pushbutton KEY(2). The maximum count is 255. Then, display the output of the counter in hexadecimal code on 7-segment displays HEX5 and HEX4. Clear the counter if pushbutton KEY(3) is pressed.

Deliverable report must contain:
- A snapshot of the final configuration of the QSYS (Platform Designer) tool.
- The top entity HDL description.
- The listings of the programs, with comments to explain the code.
- A snapshot of the Ashling RiscFree IDE for Intel FPGAs tool after one of the programs has been executed.

APPENDIX – Building a NIOS V-based system with QSYS (Platform Designer)

READ THIS VERY CAREFULLY

1. Introduction

The tutorial is intended for the DE1-SoC board (see Figure 1)

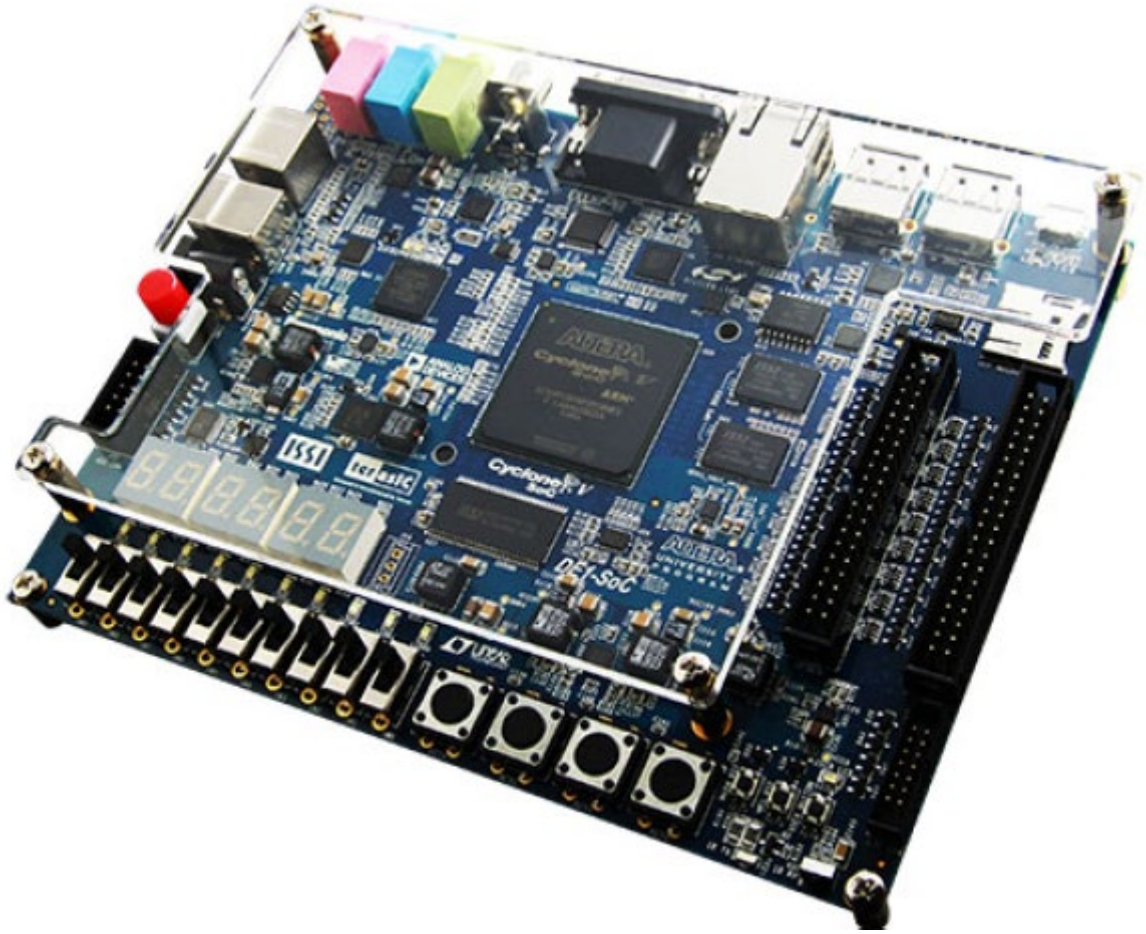


Figure 1 DE1-SoC board

This tutorial focuses on the simple hardware system depicted in Figure 2. As shown in Figure 2, the Nios V processor is connected to the memory and I/O interfaces by means of an interconnection network. This interconnection network is automatically generated by the QSYS (Platform Designer) tool. The memory component in our system will be realized by using the on-chip memory available in the FPGA chip. The I/O interfaces that connect to the slider switches and LEDs will be implemented by using the predefined modules that are available in the QSYS (Platform Designer) tool. A special JTAG UART interface is used to connect to the circuitry that provides a USB link to the host computer to which the DE-series board is connected. This circuitry and the associated software is called the USB-Blaster. Another module, called the JTAG Debug module, is provided to allow the host computer to control the Nios V system. It makes possible to perform

operations, such as downloading Nios V programs into memory, starting and stopping the execution of these programs, setting breakpoints, and examining the contents of memory and Nios V registers.

Since all parts of the Nios V system implemented on the FPGA chip are defined by using a hardware description language, a knowledgeable user could write such code to implement any part of the system. This would be an onerous and time consuming task. Instead, we will show how to use the QSYS (Platform Designer) tool to implement the desired system simply by choosing the required components and specifying the parameters needed to make each component fit the overall requirements of the system. Although in this tutorial we illustrate the capability of the QSYS (Platform Designer) tool by designing a very simple system, the same approach is used to design larger systems.

Our example system in Figure 2 is intended to realize a trivial task. Eight slider switches on the DE1-SoC board, SW7-0, are used to turn on or off eight LEDs, LEDR7-0. To achieve the desired operation, the eight-bit pattern corresponding to the state of the switches has to be sent to the output port to activate the LEDs. This will be done by having the Nios V processor execute a program stored in the on-chip memory. Continuous operation is required, such that as the switches are toggled the lights change accordingly.

In the next section we will use the QSYS (Platform Designer) tool to design the hardware depicted in Figure 2. After assigning the FPGA pins to realize the connections between the parallel interfaces and the switches and LEDs on the DE1-SoC board, we will compile the designed system. Finally, we will use the software build tool to download the designed circuit into the FPGA device, and download and execute a Nios V program that performs the desired task.

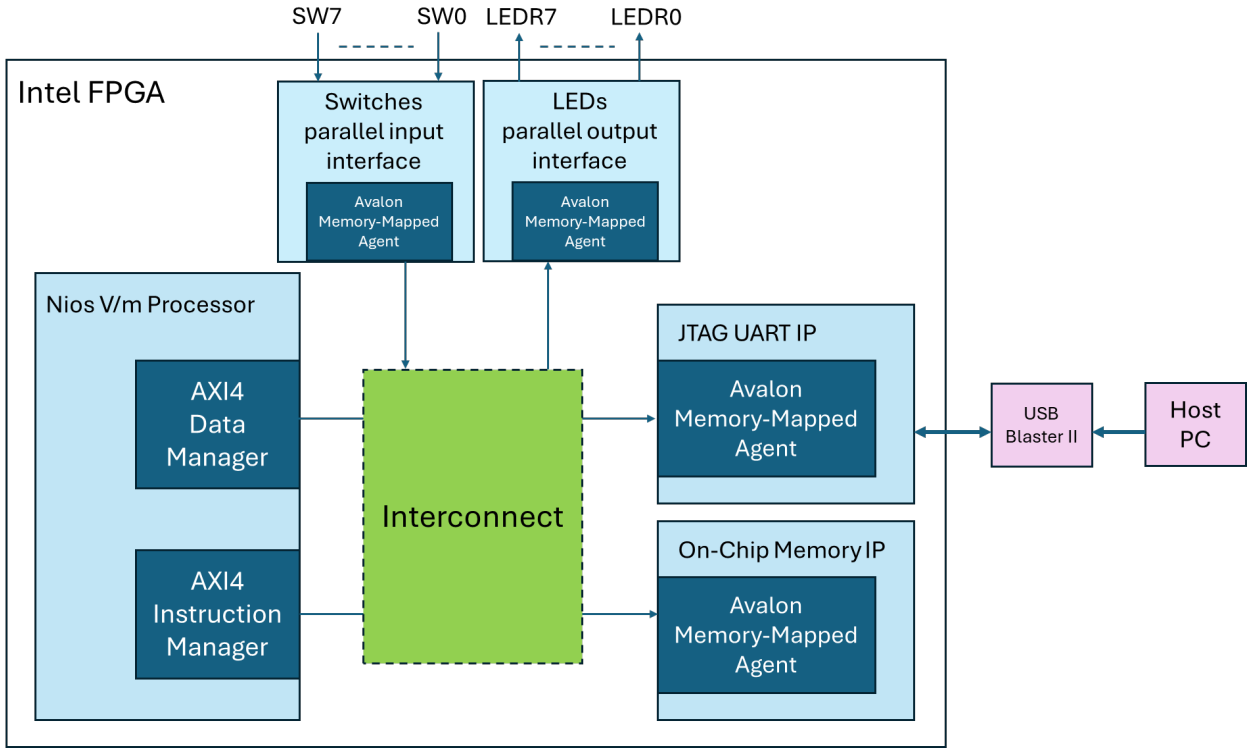


Figure 2 Simple Nios V based system

2. The QSYS (Platform Designer) tool

The QSYS (Platform Designer) tool is used in conjunction with the Quartus Prime CAD software. It allows the user to easily create a system based on the Nios V processor, by simply selecting the desired functional units and specifying their parameters.
To implement the system in Figure 2, we have to instantiate the following functional units:

- Nios V processor
- On-chip memory, which consists of the memory blocks in the FPGA chip; we will specify a 256-KiByte memory arranged in 32-bit words
- JTAG UART interface for communication with the host computer
- Two parallel I/O interfaces

To define the desired system, start the Quartus Prime software and perform the following steps:

1. Create a new Quartus Prime project for your system. As shown in Figure 3, we stored our project in a directory called *qsys_tutorial*, and we assigned the name *lights* to both the project and its top-level design entity. You can choose a different directory or project name. Step through the screen for adding design files to the project; we will add the required files later in the tutorial. In your project, choose the FPGA device used on the DE1-SoC board, namely **Cyclone V SoC 5CSEMA5F31C6.**

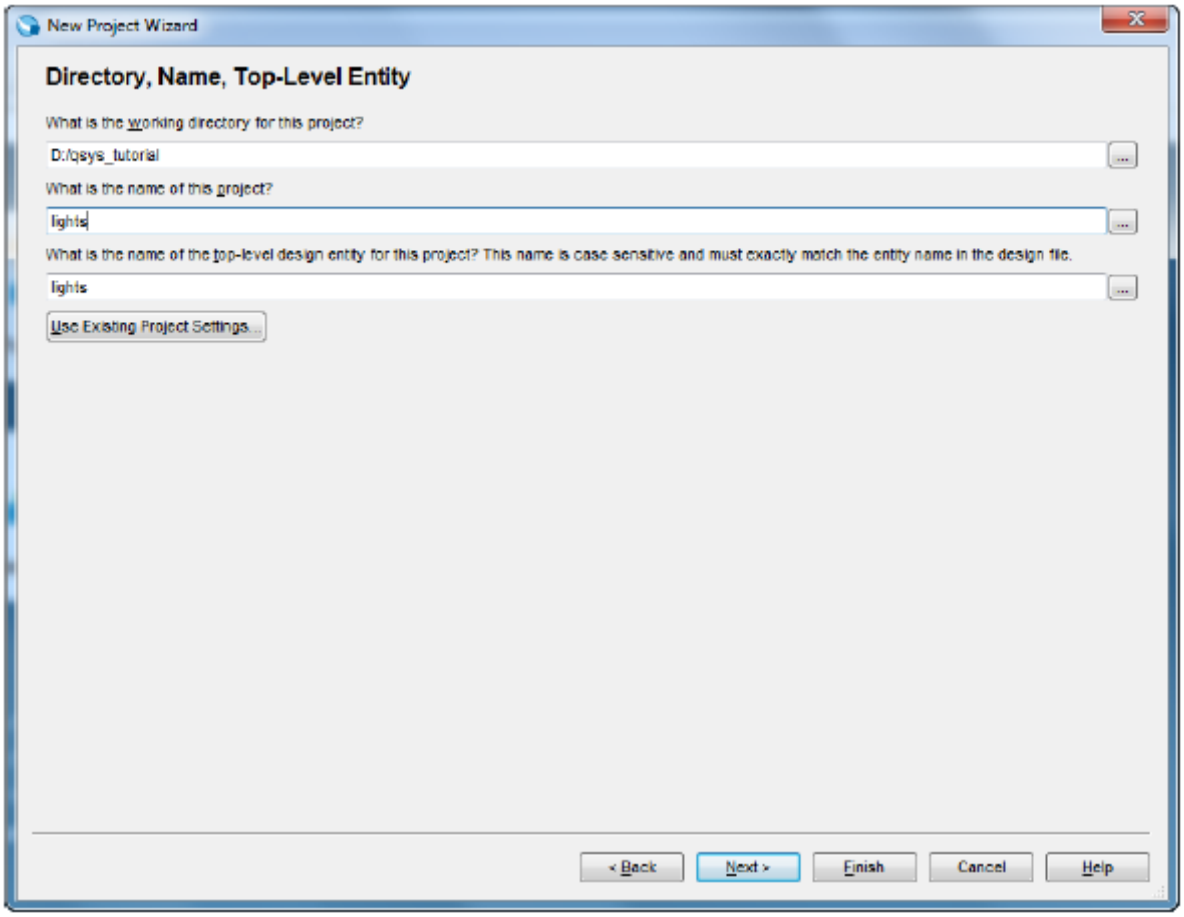


Figure 3 Create a new project

2. After completing the New ProjectWizard to create the project, in the main Quartus Prime window select **`Tools > Platform Designer`**, which leads to the window in Figure 4. This is the System Contents tab of the QSYS (Platform Designer) tool, which is used to add components to the system and configure the selected components to meet the design requirements. The available components are listed on the left side of the window.

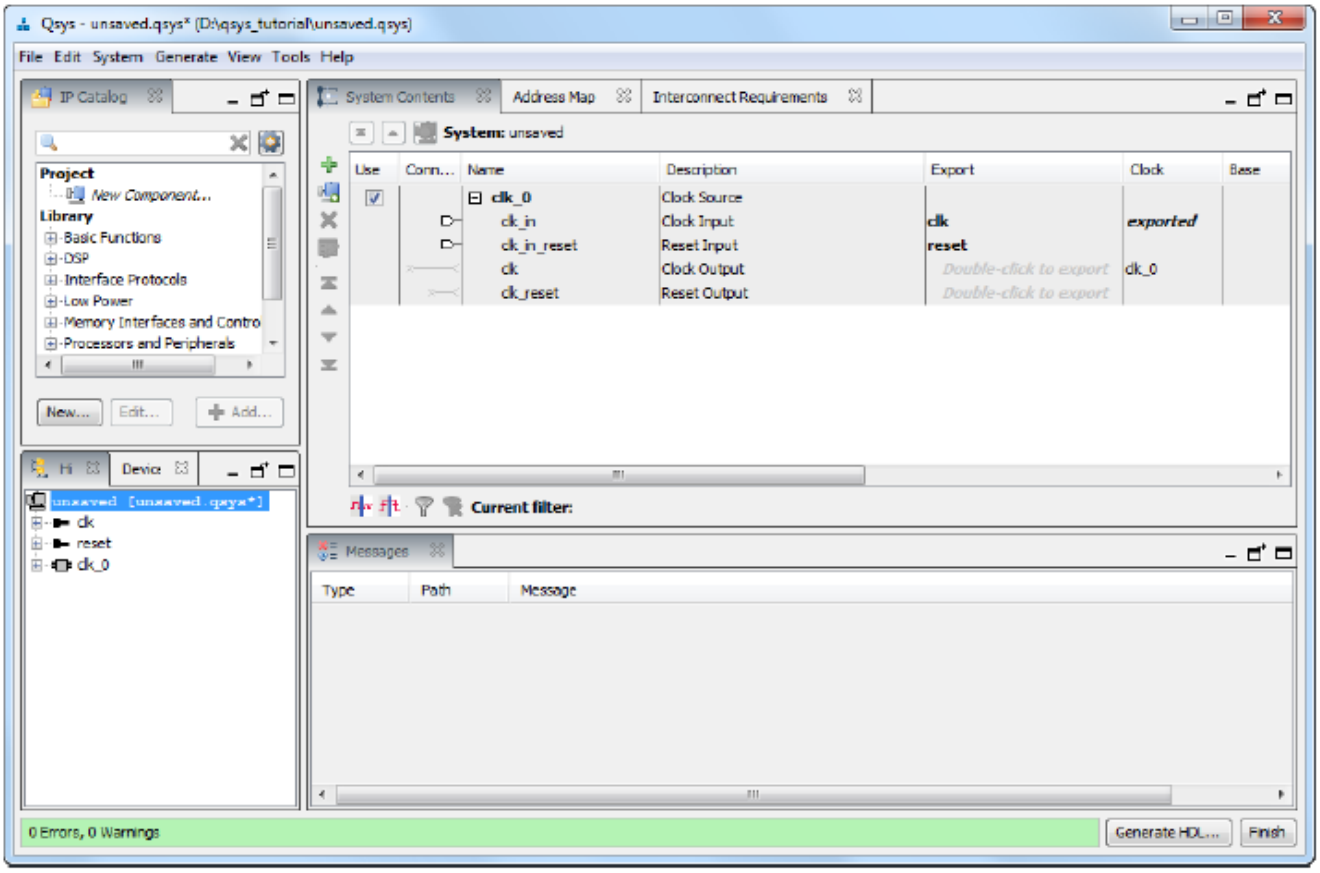


Figure 4 Create a new Nios V system

3. The hardware system that will be generated using the QSYS (Platform Designer) tool runs under the control of a clock. For this tutorial we will make use of the 50-MHz clock that is provided on the DE1-SoC board. Your hardware system should contain a clock source called *clk_0*, which frequency is 50-MHz. You can check that its frequency is indeed 50-MHz by double clicking the component, and checking the clock frequency parameter of the component. If your system does not already contain *clk_0*, it is possible to add a clock source by selecting **`Basic Functions > Clocks; PLLs and Resets > Clock Source`** in the IP Catalog tab, then clicking **`Add...`**.

4. Next, specify the processor as follows:
   - On the left side of the QSYS (Platform Designer) window expand **`Processors and Peripherals`**, select **`Embedded Processors > Nios V/m Microcontroller Intel FPGA IP`** and click **`Add...`**, which leads to the window in Figure 5.
   - The Nios V processor has reset and interrupt inputs. When one of these inputs is activated, the processor starts executing the instructions stored at memory addresses known as reset vector and interrupt vector, respectively. Since we have not yet included any memory components in our design, the QSYS (Platform Designer) tool will display corresponding error messages. Ignore these messages as we will provide the necessary information later. Click **`Finish`** to return to the main QSYS (Platform Designer) window, which now shows the Nios V processor specified as indicated in Figure 6.

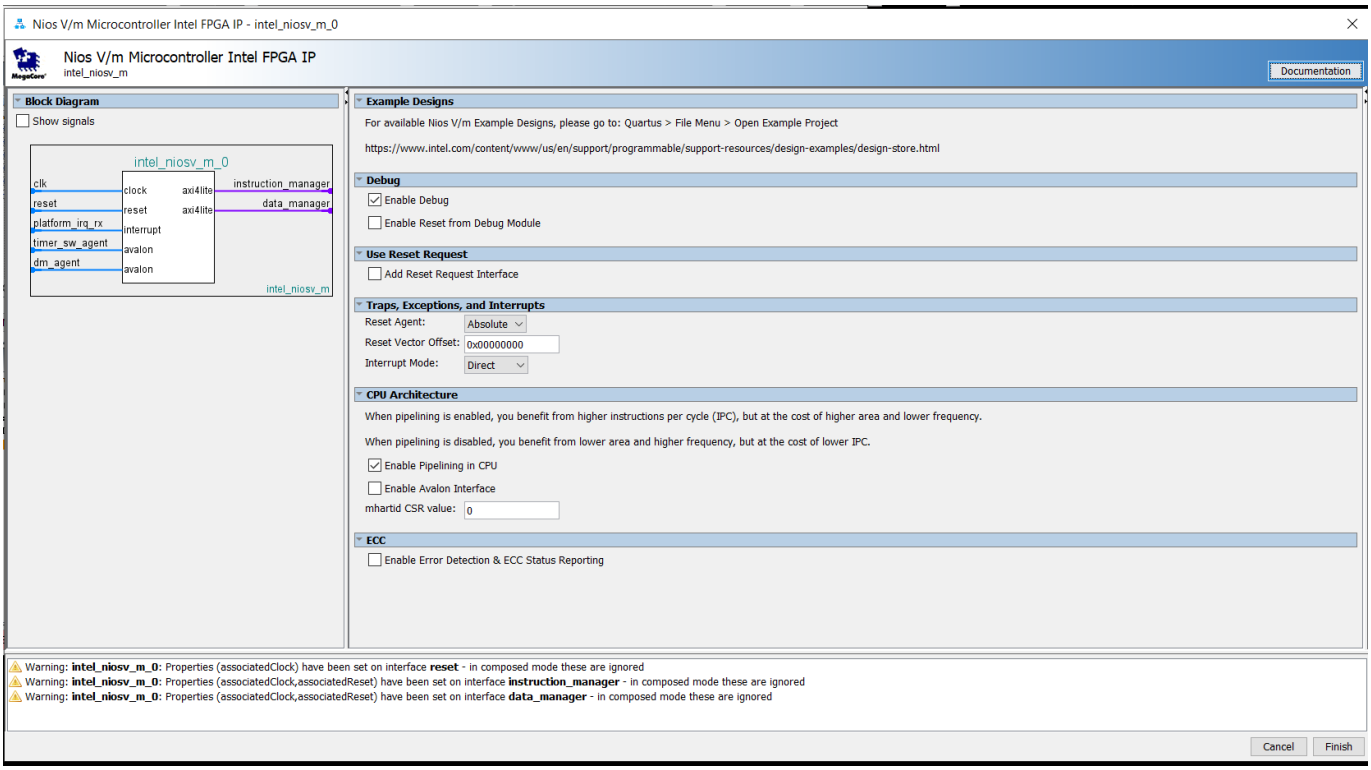


Figure 5 Create a Nios V processor

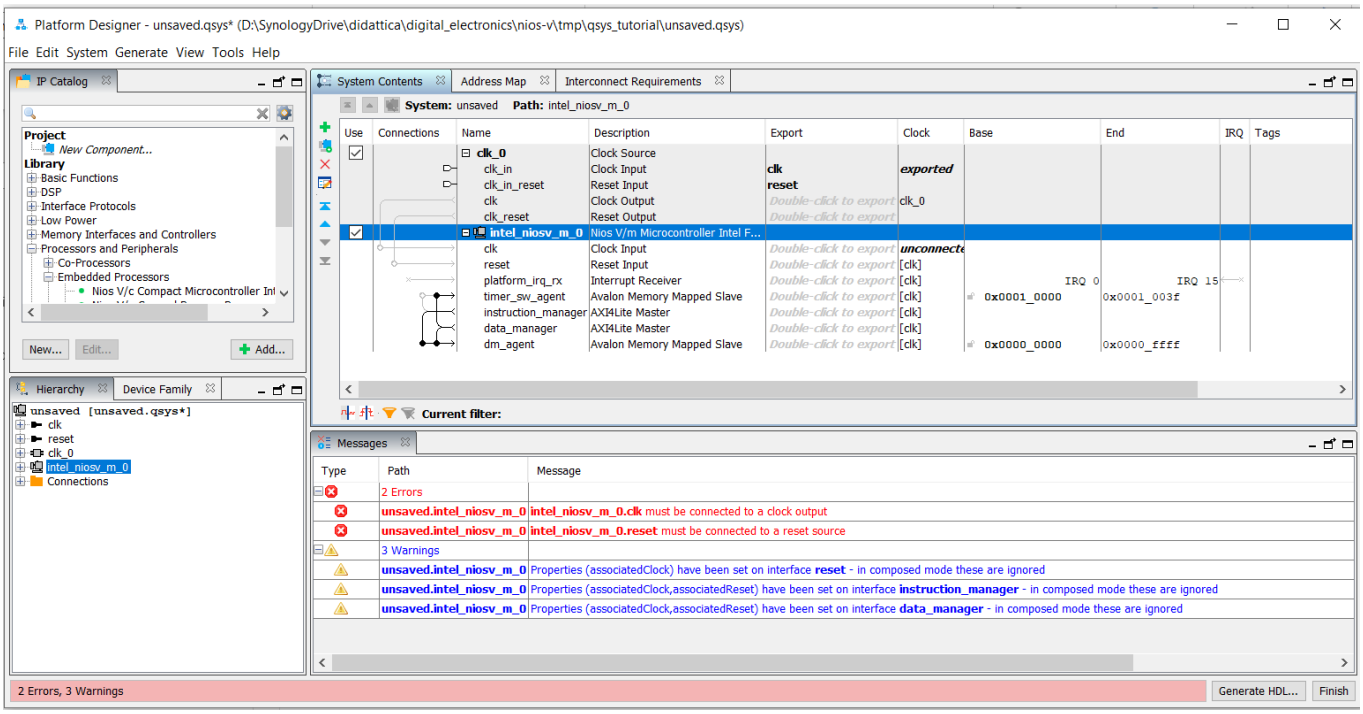


Figure 6 Inclusion of the Nios V processor in the design

5. To specify the on-chip memory perform the following:
   - Expand the category **Basic Functions**, and then expand to select **On Chip Memory > On-Chip Memory (RAM or ROM)**, and click **Add**
   - In the On-Chip Memory Configuration Wizard window, shown in Figure 7, ensure that the **Data width** is set to 32 bits and the **Total memory size** to 262144 bytes
   - Do not change the other default settings
   - Click **Finish**, which returns to the System Contents tab as indicated in Figure 8.

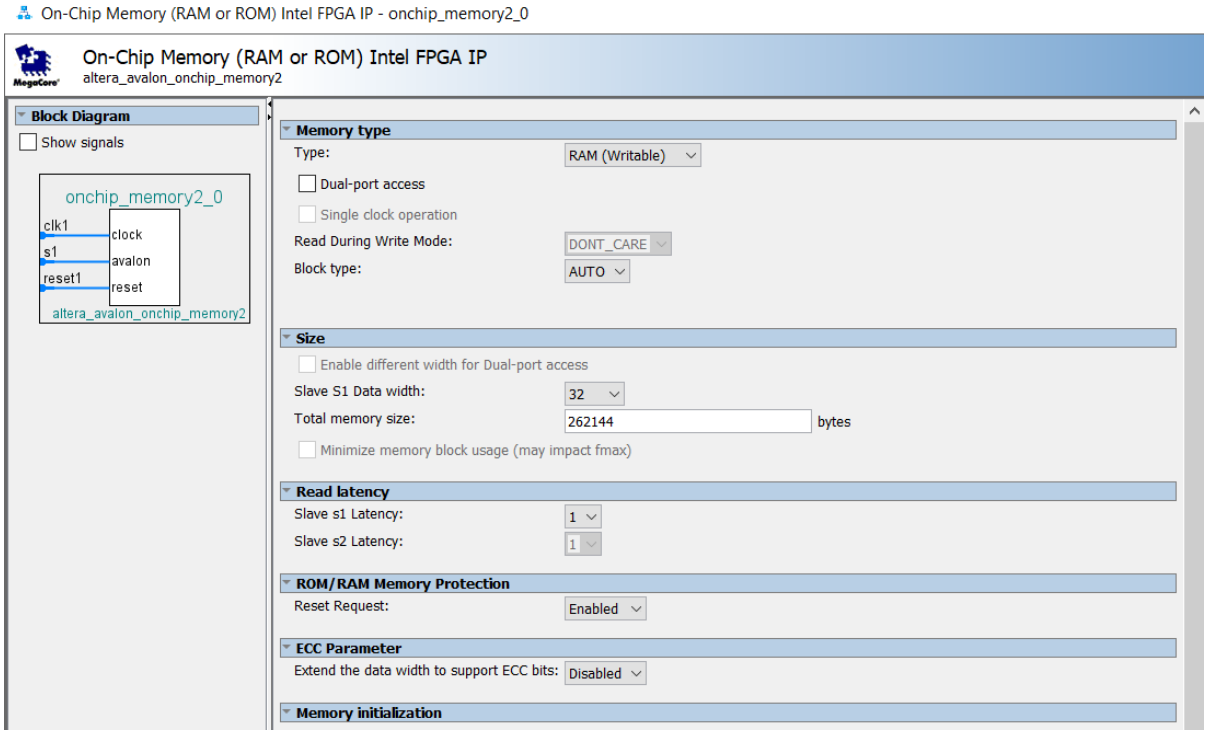


Figure 7 Define the on chip memory

6. Observe that while the Nios V processor and the on-chip memory have been included in the design, no connections between these components have been established. To specify the desired connections, examine the **Connections** area in the window in Figure 8. The connections already made are indicated by filled circles and the other possible connections by empty circles, as indicated in Figure 9. Clicking on an empty circle makes a connection. Clicking on a filled circle removes the connection. We want the clock of the memory be the same one as the processor. Thus, connect clock inputs of the processor and the memory to the clock output of the clock component. Similarly, for the reset. So connect reset inputs of the processor and the memory to the reset output of the clock component. With a Von Neuman architecture the memory contains both data and instructions: connect the *s1* input of the memory to both the *data_manager* and *instruction_manager* outputs of the processor. The resulting connections are shown in Figure 10.

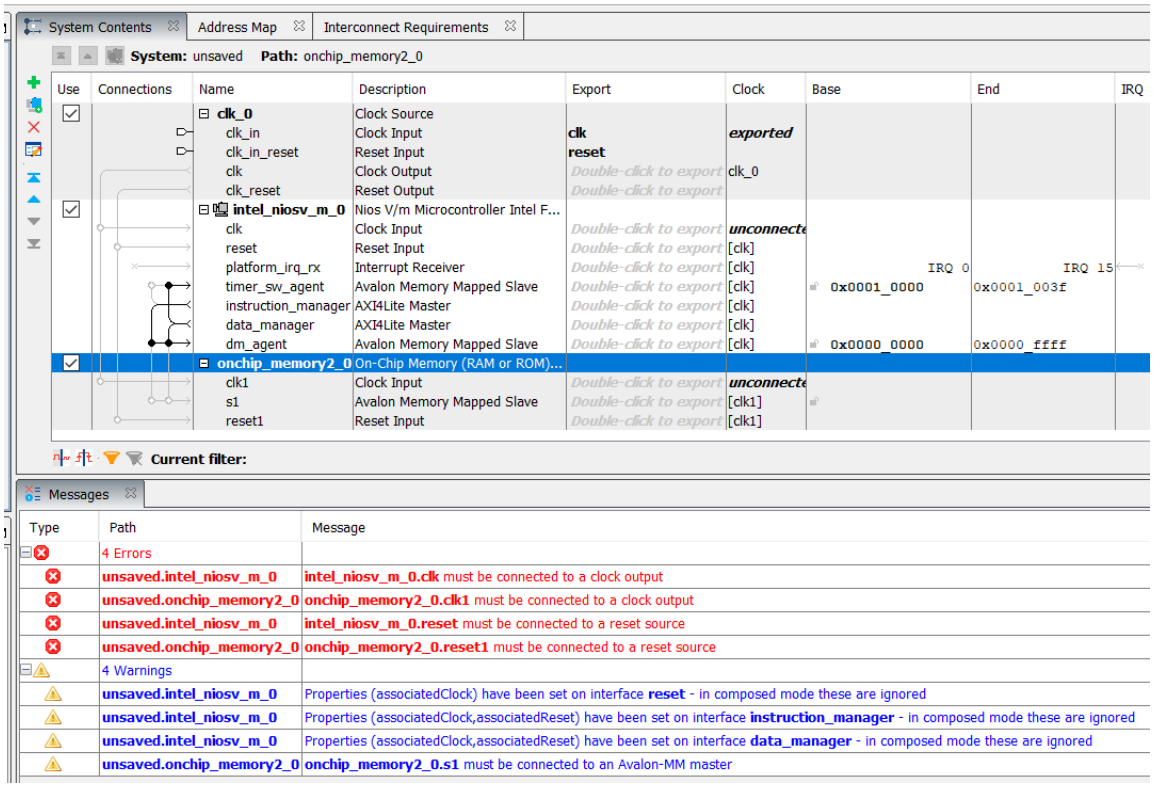


Figure 8 The on chip memory included in the design

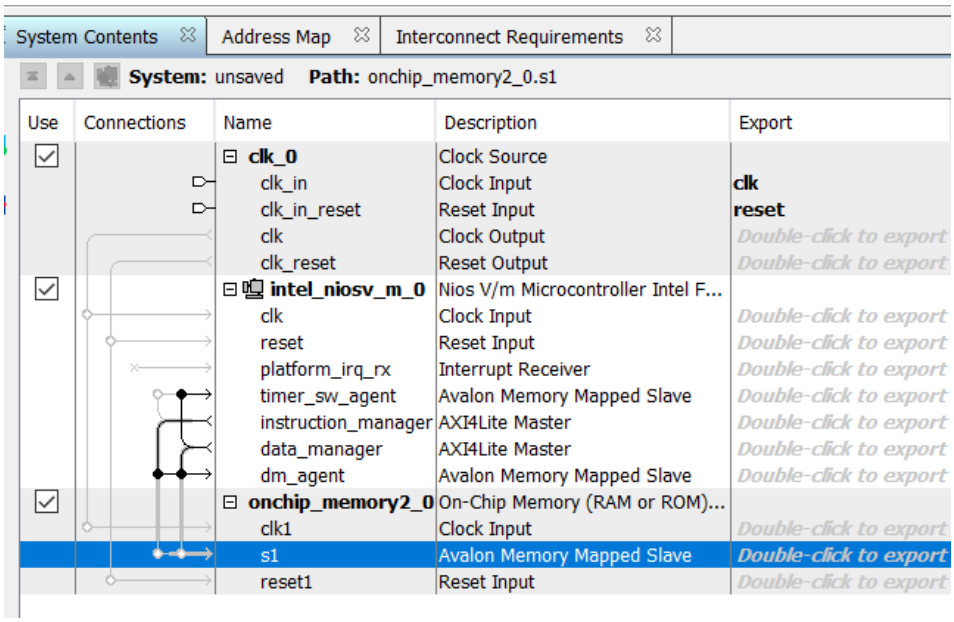


Figure 9 Connections before

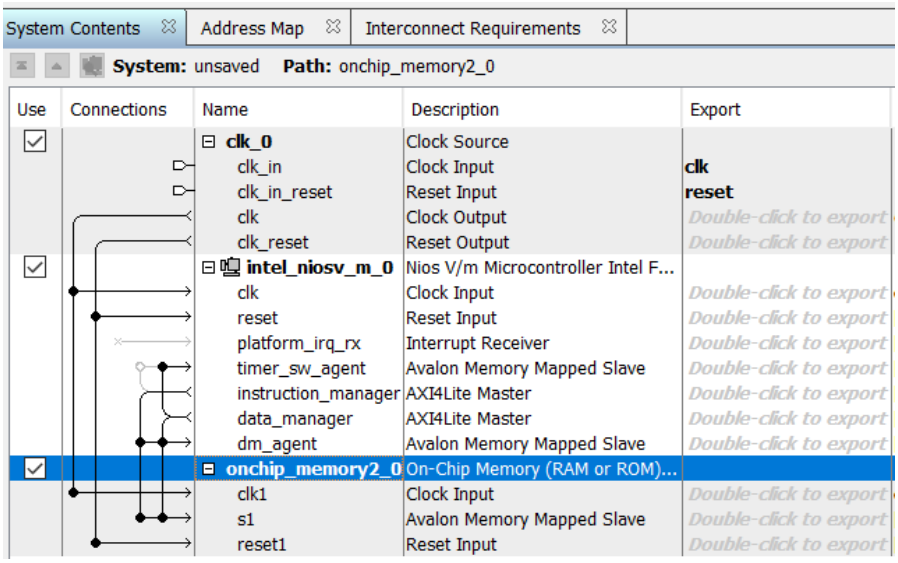


Figure 10 Connections after

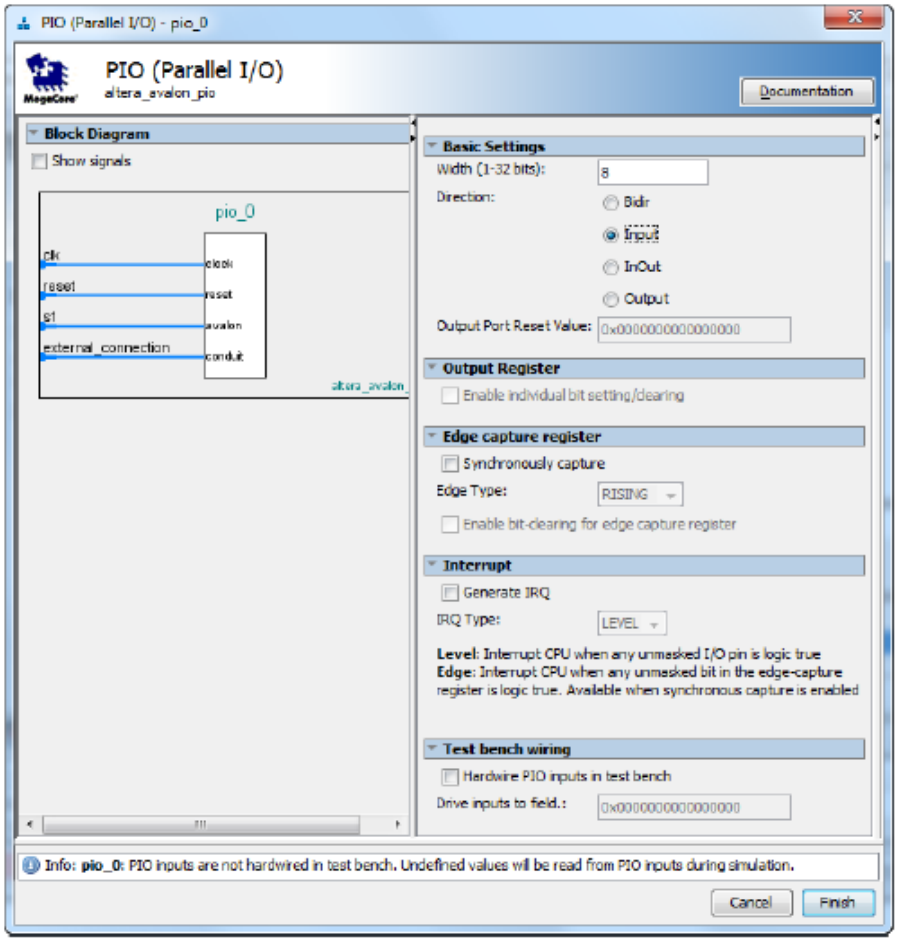


Figure 11 Define a parallel input interface

7. Specify the input parallel I/O interface as follows:
   - Select **Processors and Peripherals > Peripherals > PIO (Parallel I/O)** and click **Add** to reach the PIO Configuration Wizard in Figure 11
   - Specify the width of the port to be 8 bits and choose the direction of the port to be Input, as shown in the figure.
   - Click **Finish**.

8. In the same way, specify the output parallel I/O interface:
   - Select **Processors and Peripherals > Peripherals > PIO (Parallel I/O)** and click **Add** to reach the PIO Configuration Wizard again
   - Specify the width of the port to be 8 bits and choose the direction of the port to be Output.
   - Click **Finish** to return to the System Contents tab

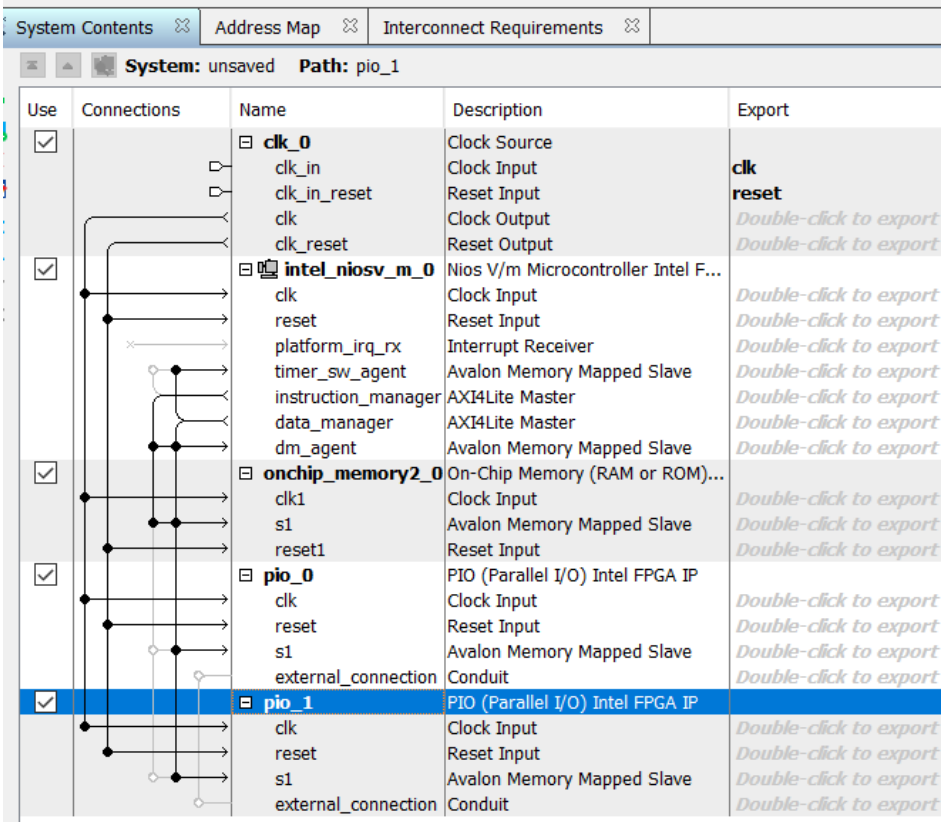


Figure 12 The system with all components and connections

9. Specify the necessary connections for the two PIOs. We need the clock of the PIOs to be same one as the processor (and the memory). Connect clock input of the PIO to the clock output of the clock component. Similarly, for the reset: connect reset input of the PIO to the reset output of the clock component. Since we want to send/receive only data (not instructions) to/from PIOs, connect the *s1* input of the PIO **only** to the *data_manager* output of the processor.
   The resulting design is depicted in Figure 12.

10. We wish to connect to a host computer and provide a means for communication between the Nios V system and the host computer. This can be accomplished by instantiating the JTAG UART interface as follows:
    - Select **Interface Protocols > Serial > JTAG UART** and click **Add** to reach the JTAG UART Configuration Wizard in Figure 13

- Do not change the default settings
- Click **`Finish`** to return to the System Contents tab

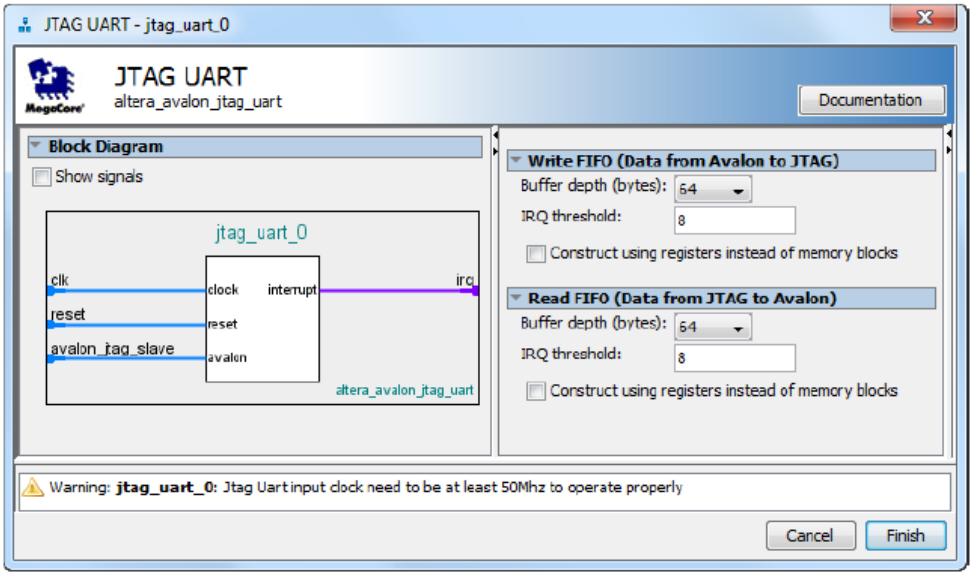


Figure 13 Define the JTAG UART interface

Connect the JTAG UART to the *clock*, *reset* and *data-manager* ports, as was done for the PIOs. Connect the Interrupt Request (*irq*) line from the JTAG UART to the Nios V processor by selecting the connection under the IRQ column, as shown in Figure 14. Once the connection is made, a box with the number 0 inside will appear on the connection. The Nios V processor has 16 interrupt ports ranging from 0 to 15, and the number in this box selects which port will be used for this IRQ. Click on the box and change it to use port 5. Make sure the *irq* port of JTAG UART gets automatically connected to the *platform_irq_rx* port of Nios V Processor.

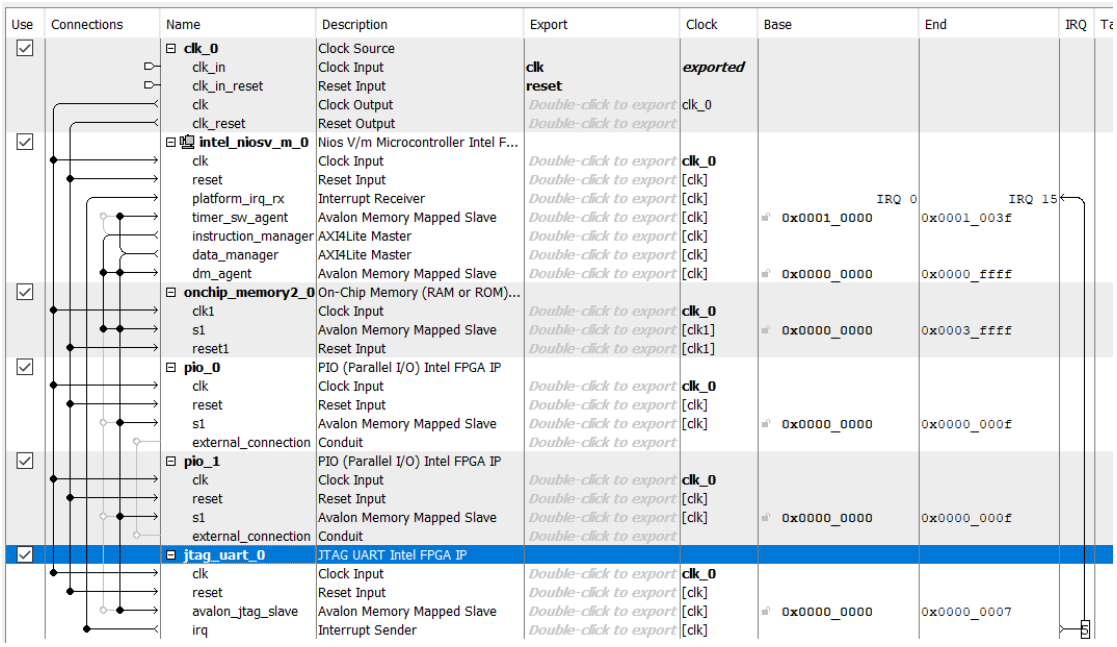


Figure 14 Connect the IRQ line from the JTAG UART to the Nios V processor

11. Note that the QSYS (Platform Designer) tool automatically chooses names for the various components. The names are not necessarily descriptive enough to be easily associated with the target design, but they can be changed. In Figure 2, we used the names *Switches* and *LEDs* for the parallel input and output interfaces, respectively. These names can be used in the implemented system. Right-click on the *pio_0* name and then select **`Rename`**. Change the name to *switches*. Similarly, change *pio_1* to *LEDs*. Figure 15 shows the system with name changes that we made for all components.

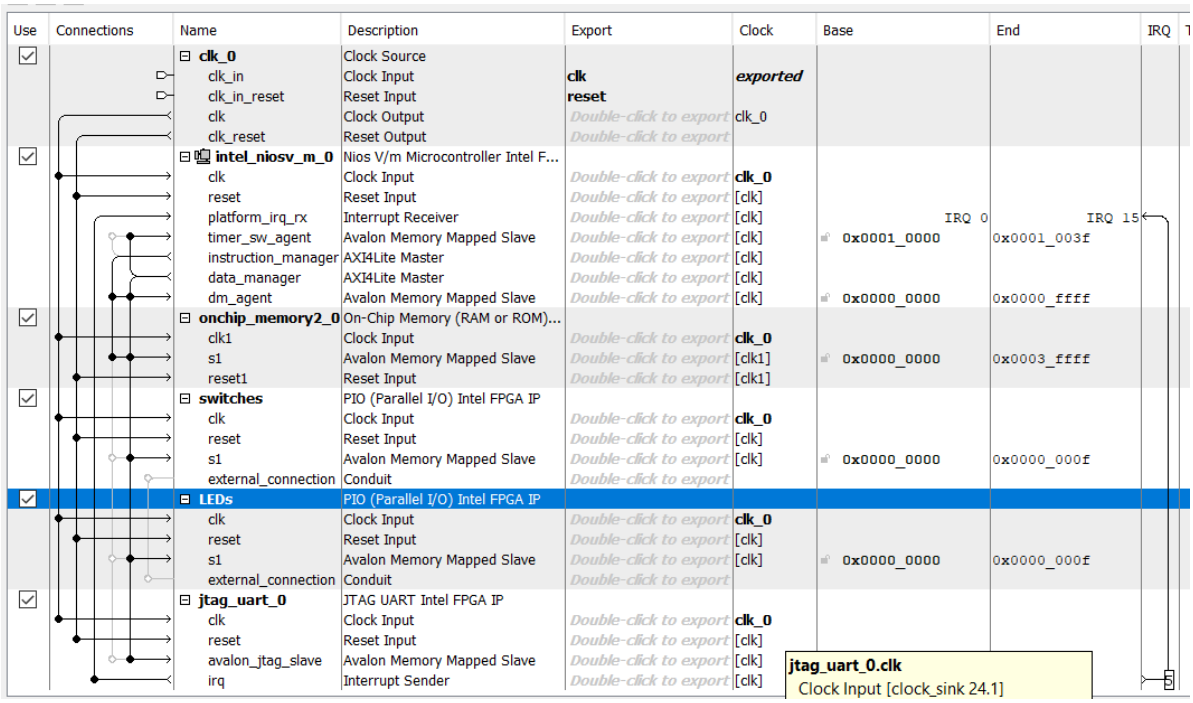


Figure 15 The system with all components appropriately named

12. Observe that the base and end addresses of the various components in the designed system have not been properly assigned. These addresses can be assigned by the user, but they can also be assigned automatically by the QSYS (Platform Designer) tool. We will choose the latter possibility. However, we want to make sure that the on-chip memory has the base address of zero. Double-click on the Base address for the on-chip memory in the QSYS (Platform Designer) window and enter the address 0x00000000. Then, lock this address by clicking on the adjacent lock symbol. Now, let QSYS (Platform Designer) assign the rest of the addresses by selecting **`System > Assign Base Addresses`** (at the top of the window), which produces an assignment similar to that shown in Figure 16.

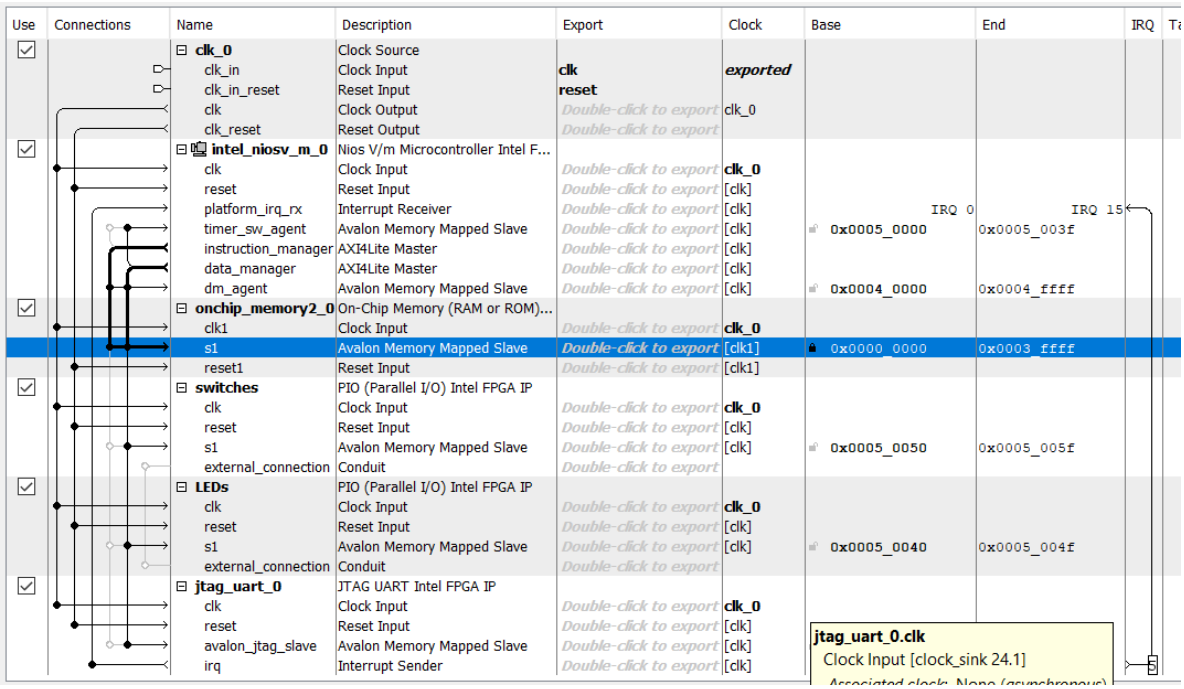


Figure 16 The system with assigned addresses

13. The behavior of the Nios V processor when it is reset is defined by its reset vector. It is the location in the memory device from which the processor fetches the next instruction when it is reset. Similarly, the exception vector is the memory address of the instruction that the processor executes when an interrupt is raised. To specify these two parameters, perform the following:
    - Right-click on the *intel_niosv_m0* component in the window displayed in Figure 16, and then select **`Edit`** to reach the window in Figure 17
    - Select *onchip_memory2_0.s1* to be the memory device (Reset Agent) for Traps, Exceptions and Interrupts, as shown in Figure 17
    - Do not change the default settings for offsets
    - Click **`Finish`** to return to the System Contents tab

14. So far, we have specified all connections inside our *nios_system* circuit. It is also necessary to specify connections to external components, which are switches and LEDs in our case. To accomplish this, double click on **`Double-click to export`** (in the Export column of the System Contents tab) for **`external_connection`** of the switches PIO, and type the name *switches*. Similarly, establish the external connection for the lights, called *leds*. This completes the specification of our *nios_system*, which is depicted in Figure 18.

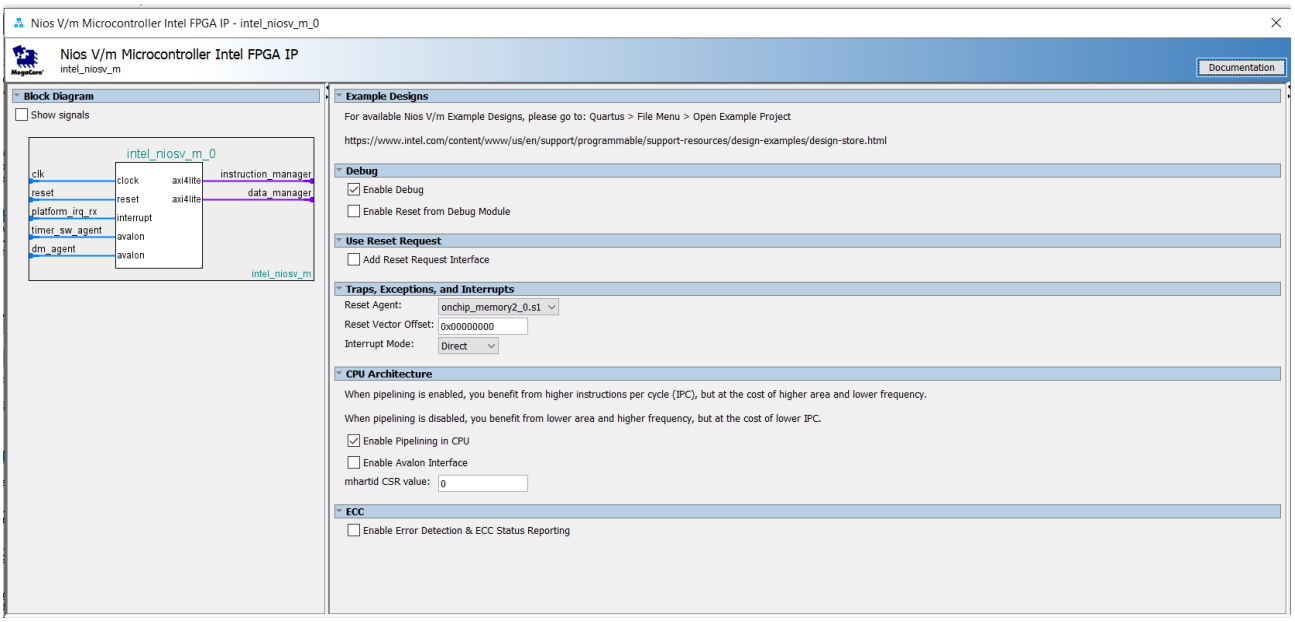


Figure 17 Define the Traps, Exceptions and Interrupts Agent

15. Having specified all components needed to implement the desired system, it can now be generated. Save the specified system; we used the name *nios_system*. Then, select **`Generate > Generate HDL`**, which leads to the window in Figure 19. Select **`None`** for the option **`Simulation > Create simulation model`**, because in this tutorial we will not deal with the simulation of hardware. Click **`Generate`** on the bottom of the window. When successfully completed, the generation process produces the message “Generate Completed".

    Exit the QSYS (Platform Designer) tool to return to the main Quartus Prime window.

    Changes to the designed system are easily made at any time by reopening the QSYS (Platform Designer) tool. Any component in the System Contents tab of the QSYS (Platform Designer) tool can be selected and edited or deleted, or a new component can be added and
    the system regenerated.

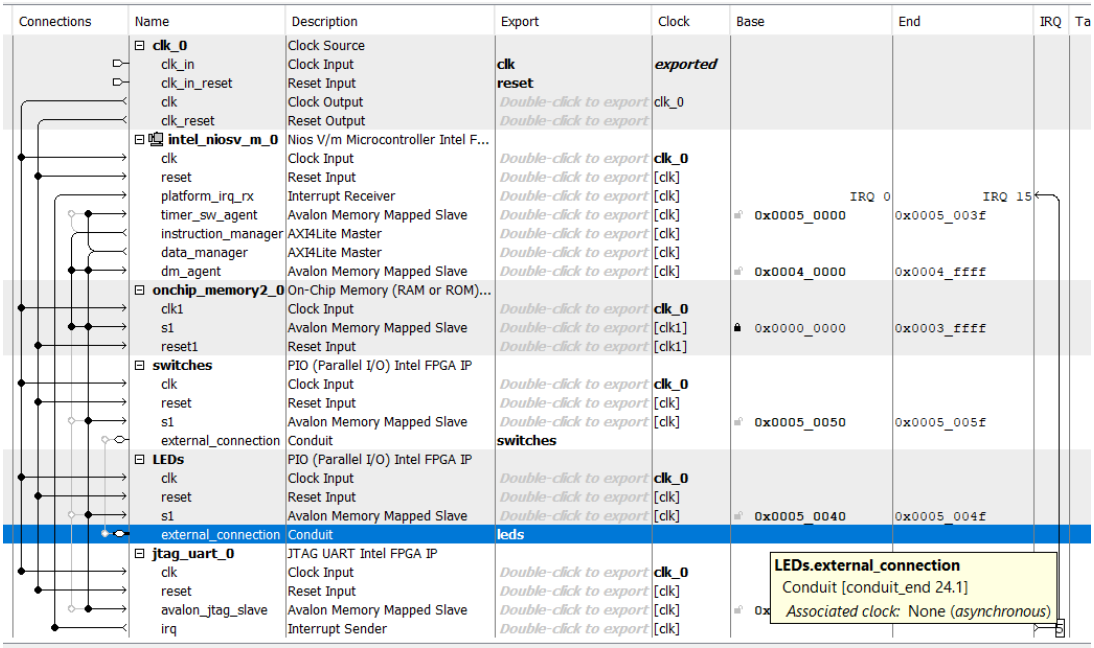


Figure 18 The complete system

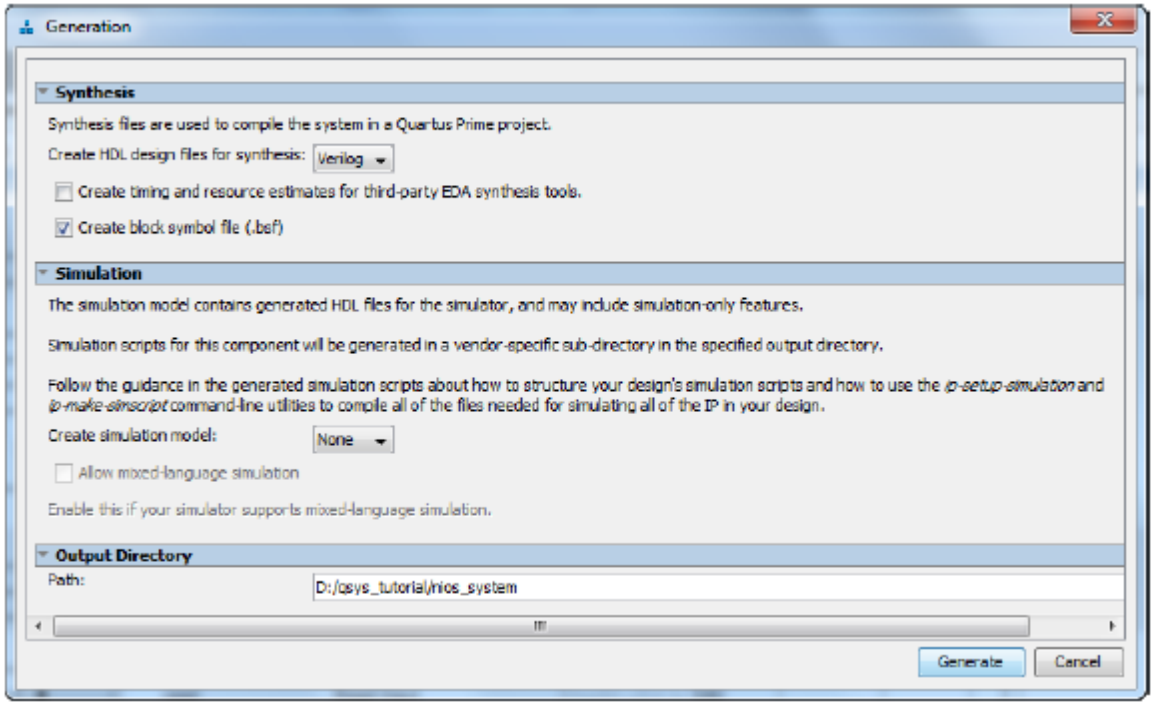


Figure 19 Generation of the system

3. Integration of the Nios V System into a Quartus Prime Project

To complete the hardware design, we have to perform the following:

- Instantiate the module generated by the QSYS (Platform Designer) tool into the Quartus Prime project
- Assign the FPGA pins
- Compile the designed circuit
- Program and configure the FPGA device on the DE1-SoC board

3.1 Instantiation of the Module Generated by the QSYS (Platform Designer) Tool

The QSYS (Platform Designer) tool generates a Verilog/VHDL module that defines the desired Nios V system. In our design, this module will have been generated in the *nios_system.v*/*nios_system.vhd* file, which can be found in the directory *qsys_tutorial/nios_system/synthesis* of the project. The QSYS (Platform Designer) tool generates Verilog/VHDL modules, which can then be used in designs specified using either Verilog or VHDL languages.

Normally, the Nios V module generated by the QSYS (Platform Designer) tool is likely to be a part of a larger design. However, in the case of our simple example there is no other circuitry needed. All we need to do is instantiate the Nios V system in our top-level Verilog or VHDL module, and connect inputs and outputs of the parallel I/O ports, as well as the clock and reset inputs, to the appropriate pins on the FPGA device.

The Verilog/VHDL code in the *nios_system.v*/*nios_system.vhd* file is quite large. Figure 20 depicts the portion of the code that defines the input and output ports for the module *nios_system*. The 8-bit vector that is the input to the parallel port switches is called *switches_export*. The 8-bit output vector is called *leds_export*. The clock and reset signals are called *clk_clk* and *reset_reset_n*, respectively. Note that the reset signal was added automatically by the QSYS (Platform Designer) tool; it is called *reset_reset_n* because it is active low.

```
// Verilog
module nios_system (
    input        clk_clk,          // clk.clk
    output [7:0] leds_export,      // leds.export
    input        reset_reset_n,    // reset.reset_n
    input  [7:0] switches_export   // switches.export
);
Endmodule


-- VHDL
entity nios_system is
        port (
                clk_clk         : in  std_logic                     := '0';             --      clk.clk
                leds_export     : out std_logic_vector(7 downto 0);                     --     leds.export
                reset_reset_n   : in  std_logic                     := '0';             --    reset.reset_n
                switches_export : in  std_logic_vector(7 downto 0) := (others => '0')  -- switches.export
        );
end entity nios_system;
```

Figure 20 A part of the generated Verilog/VHDL

The *nios_system* module has to be instantiated in a top-level module that has to be named *lights*, because this is the name we specified in Figure 3 for the top-level design entity in our Quartus Prime project. For the input and output ports of the *lights* module we have used the pin names that are specified in the DE1-SoC User Manual: *CLOCK_50* for the 50-MHz clock, *KEY* for the pushbutton switches, *SW* for the slider switches, and *LEDR* for the red LEDs.
Using these names simplifies the task of creating the needed pin assignments, as they are the default names in the *DE1_SoC.qsf* file.

Figure 21 shows a top-level Verilog/VHDL module that instantiates the Nios V system. Instantiate this module into a file called *lights.v/lights.vhd*.

```
// Verilog
module lights (
    input        CLOCK_50,
    input  [0:0] KEY,
    input  [7:0] SW,
    output [7:0] LEDR
);

// Instantiate the nios_system module
nios_system NiosV (
    .clk_clk(CLOCK_50),
    .reset_reset_n(KEY[0]),
    .switches_export(SW),
    .leds_export(LEDR)
);

endmodule

-- VHDL
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY lights IS
PORT (
       CLOCK_50 : IN STD_LOGIC;
       KEY : IN STD_LOGIC_VECTOR (0 DOWNTO 0);
       SW : IN STD_LOGIC_VECTOR (7 DOWNTO 0);
       LEDR : OUT STD_LOGIC_VECTOR (7 DOWNTO 0));
END lights;

ARCHITECTURE lights_rtl OF lights IS

COMPONENT nios_system
       PORT (
              clk_clk: IN STD_LOGIC;
              reset_reset_n : IN STD_LOGIC;
              switches_export : IN STD_LOGIC_VECTOR (7 DOWNTO 0);
              leds_export : OUT STD_LOGIC_VECTOR (7 DOWNTO 0));
END COMPONENT;

BEGIN

NiosV : nios_system
       PORT MAP(
              clk_clk => CLOCK_50,
              reset_reset_n => KEY(0),
              switches_export => SW(7 DOWNTO 0),
              leds_export => LEDR(7 DOWNTO 0));

END lights_rtl;
```

Figure 21 Instantiating the Nios V system using Verilog/VHDL code

4. Compiling the Quartus Prime Project

Add the *lights.v/lights.vhd* file to your Quartus Prime project. Also, add the necessary pin assignments for the DE1-SoC board (*DE1_SoC.qsf*). The procedure for making pin assignments is described in the LAB#0 document.

Since the system we are designing needs to operate at a 50-MHz clock frequency, we can add the needed timing assignment in the Quartus Prime project. The tutorial Using TimeQuest Timing Analyzer shows how this is done (see LAB#2 appendix). Finally, before compiling the project, it is necessary to add the *nios_system.qip* file (IP Variation file) in the directory *QSYS (Platform Designer)_tutorial/nios_system/synthesis* to your Quartus Prime project. Then, compile the project. You may see some warning messages associated with the Nios V system, such as some signals being unused or having wrong bit-lengths of vectors; these warnings can be ignored.

Note: the generation of the bistream (.sof file) depends on the availability of a Nios V license, which can be downloaded for free from Altera website (https://www.altera.com/SSLC)..
Once you have the license you can simply read it in Quartus Prime: **`Tools > License Setup`** click the dots in the *License file* field and select the license you downloaded from Altera website.

5. Ashling RiscFree IDE

Having configured the required hardware in the FPGA device, it is now necessary to create and execute an application program that performs the desired operation. Such a program is composed of a user defined part and a system defined support library. In order to perform the execution the program requires:

- to be edited
- to be compiled
- to be linked
- to be transferred to target system and launched

These tasks are supported by the Ashling RiscFree IDE, an easy-to-use GUI that automates build and execution management by integrating also a text editor and a debugger. Software application templates included in the GUI make it easy for new software programmers to get started quickly. We will use the Ashling RiscFree IDE to compile a simple C language example software program to run on the Nios V standard system configured in the FPGA on the DE1-SoC board.

To ensure a streamlined build flow, you are encouraged to create similar directory tree in your design project. The following software design flow is based on this directory tree. To create the software project directory tree, follow these steps:

a. In your design project folder (*qsys_tutorial*), create a folder called *software*.
b. In the *software* folder, create two folders called *app* and *bsp* respectively.

1. Create a Board Support Package (BSP) project

Board Support Package (BSP) provides a software runtime environment for embedded systems, such as Nios V/m processor systems. QSYS (Platform Designer) includes the BSP Editor tool, to generate and configure BSP contents. From the installation path of Quartus Prime, in *niosv/bin* run `niosv-shell`.

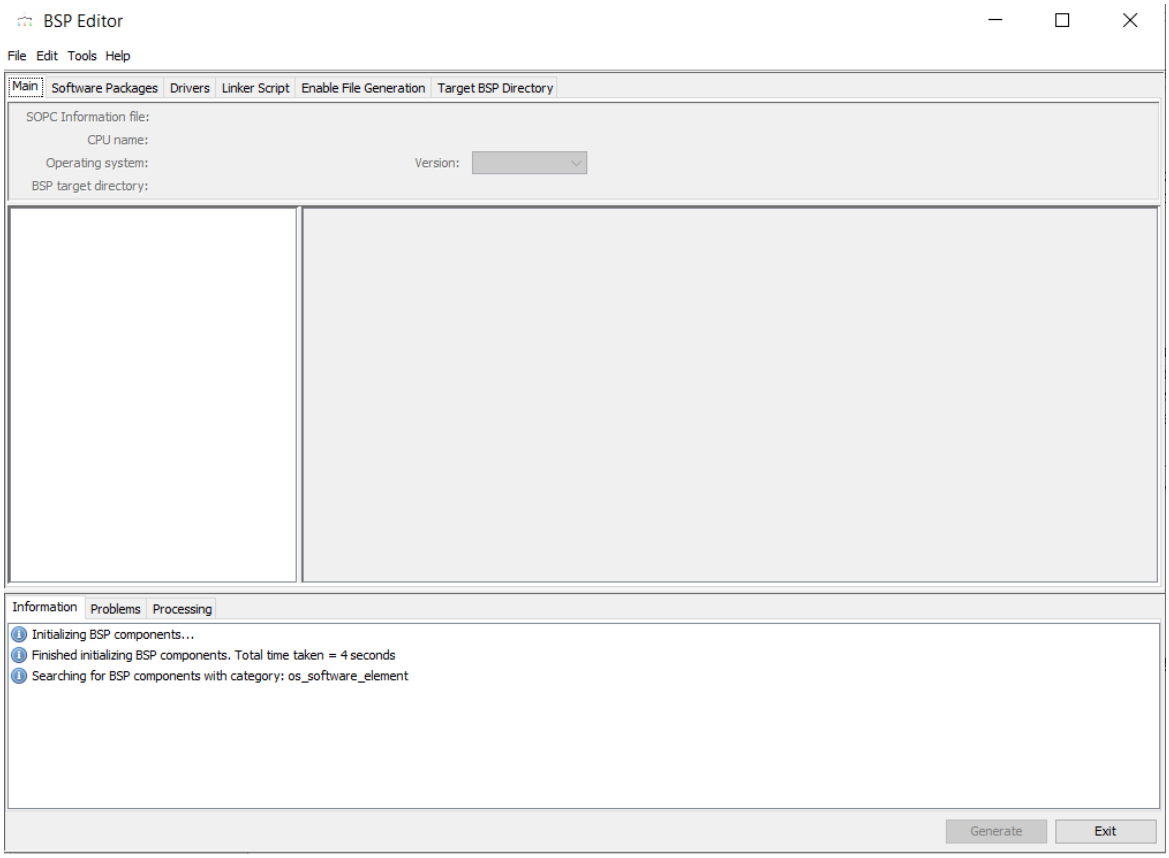


Figure 22 BSP Editor

Change the current directory to *qsys_tutorial*. Then, in the shell invoke the BSP editor as: `niosv-bsp-editor`. A window pops up (see Figure 22). Go to **`File > New BSP.`**

a. The *Create New BSP* window appears.
b. For SOPC Information File name, select the system created with QSYS (Platform Designer) that in our case is named *nios_system.sopcinfo*.
c. For CPU name, select *intel_niosv_m_0*.
d. For Operating system, select Altera HAL.
e. Unselect the *Use defaults locations* box and change the BSP destination folder from *hal_bsp* to *bsp*
f. Click *OK*, the BSP Editor tab appears.
g. This example design uses the default BSP settings, so you do not need to modify further. Click *Generate* to generate the BSP file. The BSP Editor generates the BSP files in *qsys_tutorial/software/bsp* folder. Click *Exit* to close the BSP Editor.

2. Create a Nios V processor application project with Hello World source code

In the *qsys_tutorial/software/app* folder create a source file named *hello.c*. The content of the file is shown in Figure 23

```
#include <stdio.h>
#include <unistd.h>

void looper()
{
  for (int i = 0; i < 1000; ++i)
    {
      printf("Hello world, this is the Nios V/m cpu checking in %d...\n",
i);
    }
}

int main()
{
  looper();
  usleep(1000000);
  printf("Bye world!\n");
  fflush(stdout);
  return 0;
}
```

Figure 23 Hello World example code

From the `niosv-shell` run:

```
niosv-app --bsp-dir=software/bsp \
          --app-dir=software/app \
          --srcs=software/app/hello.c \
          --elf-name=hello.elf
```

This command generates the *CMakeLists.txt* file in the *app* folder.

3. Import both BSP and application projects into RiscFree IDE's workspace.

From the installation path of Quartus Prime in the *riscfree/RiscFree* folder run `RiscFree`. The Ashling RiscFree IDE for Intel FPGAs opens.

a. Set the *qsys_tutorial* folder as the workspace.

b. Open the Import wizard (**`File > Import Nios V CMake Project`**). Browse and select the location of the Nios V processor BSP project (*software/bsp*). The project name is filled automatically. Click Finish.
c. Open the Import wizard (**`File > Import Nios V CMake Project`**). Browse and select the location of the Nios V processor APP project (*software/app*). The project name is filled automatically. Click Finish.

4. Run the example code

a. Compile the code

In order to run the example code, you have first to compile it. From the `niosv-shell` run:

```
cmake -G "Unix Makefiles" -DCMAKE_BUILD_TYPE=Debug -B software/app/debug -S software/app
make -C software/app/debug
```

These two commands create proper Makefiles and then invoke the C compiler to i) compile the BSP and generate a library, ii) compile the application (hello.c) and link the library to obtain the executable (.elf) file. Now you can run the program on Nios V.

b. Download the software ELF file

From the Ashling RiscFree IDE for Intel FPGAs select **`Run > Run Configurations`** and in the new window double click *Ashling RISC-V Hardware Debugging* and name it as *ELF_download*. Then, in the Main tab, make set the field Project as *app* and in C/C++ Application browse and choose *hello.elf* (*qsys_tutorial/software/app/debug/hello.elf*) as in Figure 24.

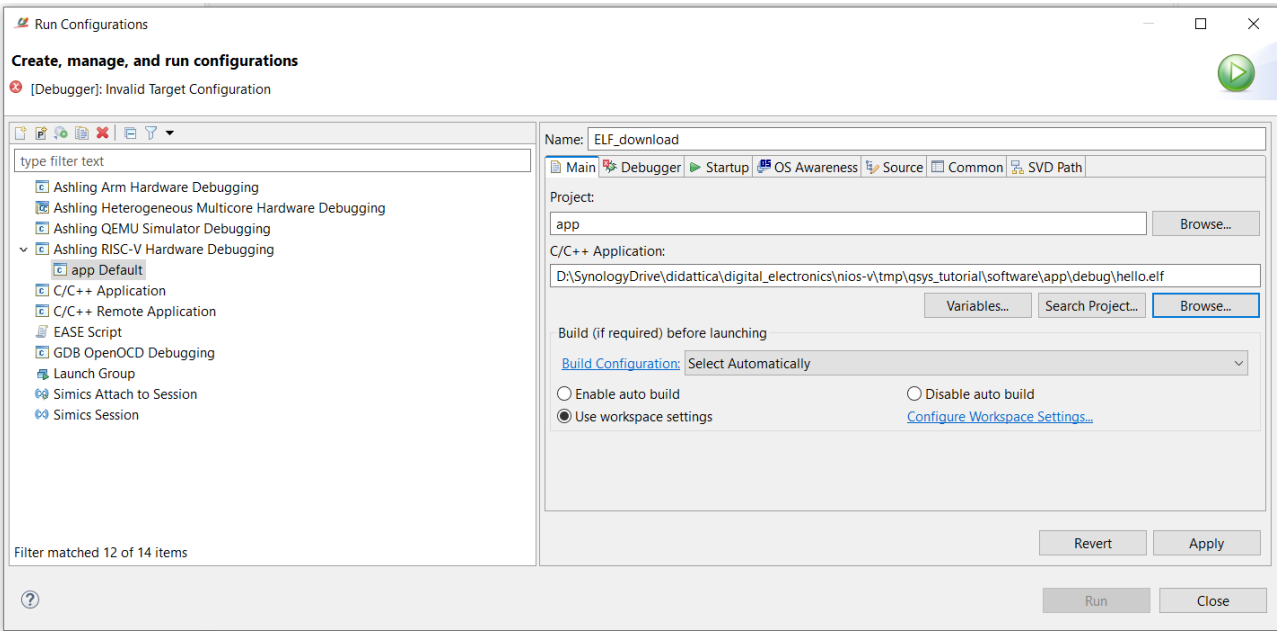


Figure 24 Configuration Main tab

Then, in the Debugger tab:

1. verify the Debug Probe Configuration settings:
   i. Debug probe: DE-SoC [USB-1]
   ii. Transport type: JTAG
   iii. JTAG frequency: 16 MHz

2. in the Target Configuration section:

i. Click Auto-detect Scan Chain to list all possible cores.
ii. Select 5CSE (BA5|MA5) as Device/TAP and Nios V as Core.

The result should be the one shown in Figure 25. Click *Apply* and *Run*.

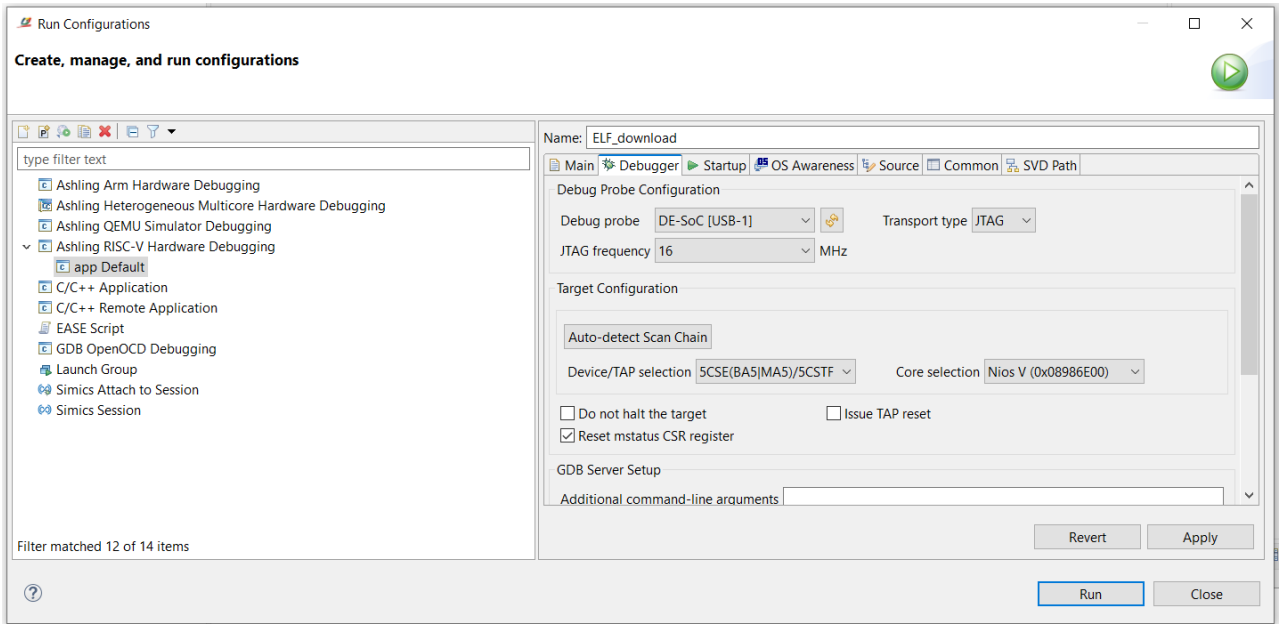


Figure 25 Configuration Debugger tab

c. Apply External Tool Configuration and execute application

External tool configuration is a generic feature where you can configure the *Ashling RiscFree IDE* for Intel FPGAs to include external tools. An example of such tools is the `juart-terminal` executable, which is used to display the Hello World message through the JTAG UART IP. To perform external tool configuration for `juart-terminal`, follow these steps:

I. Go to **`Run > External Tools > External Tools Configurations`**.
II. Double click *Program* to open a New_configuration window.
III. Rename the configuration as *Nios_V_JTAG_UART_Output*.
IV. In *Location*, click *Browse File system*.
V. Browse and select the `juart-terminal` file in the Quartus Prime installation path by locating the *quartus/bin64* folder.
VI. Set the Arguments as `-c 1 -d 0 -i 0`. This configures the JTAG UART connection towards the JTAG UART IP at cable 1, device 0, and instance 0. NOTE: JTAG device chain index varies according to your setup. Run `jtagconfig -debug` to obtain the latest device chain index numbering as you might have to update the parameters.
VII. Click Apply and click Run.
VIII. Check that the target is connected correctly. You should see in the *Console* the message:

```
juart-terminal: connected to hardware target using JTAG UART on cable
juart-terminal: "DE-SoC [USB-1]", device 2, instance 0
juart-terminal: (Use the IDE stop button or Ctrl-C to terminate)
```

followed by the Hello world message.

5. Customize and run your source code

First stop the execution of the example by clicking on the red square (**Terminate**) on the right side of the *Nios V Console* tab. Then, we can modify the source code.

What the new code has to do is:

- read the value on the input PIO (*SWITCHES*)
- write the value on the output PIO (*LEDS*)

This can be obtained by using the `IORD` and `IOWR` functions of the PIOs, namely `IORD_ALTERA_AVALON_PIO_DATA` and `IOWR_ALTERA_AVALON_PIO_DATA`, which are defined in the *altera_avalon_pio_regs.h* header file. Thus, the first modification is to include this header file. You can open the header file to see the two functions by clicking on **bsp > drivers > inc > altera_avalon_pio_regs.h**

The `IORD_ALTERA_AVALON_PIO_DATA(base)` function requires the base address of the PIO. This information has been defined into QSYS (Platform Designer) when building the system. However, we can exploit the *system.h* header file, which contains mnemonic symbols associated to the base address of each peripheral, instead of writing the base address value. As a consequence, we have first to include the *system.h* header file and then refer to the base address of the *SWITCHES* PIO simply as *SWITCHES_BASE*.

Similarly, for the `IOWR_ALTERA_AVALON_PIO_DATA(base,data)` we have to specify the base address of the PIO (*LEDS_BASE*) and the value we want to write to the LEDs.

The final code is shown in Figure 26.

```
#include "system.h"
#include "sys/alt_stdio.h"
#include "altera_avalon_pio_regs.h"


int main ()
{
      int flag;

      alt_putstr("Led and switch test\n");

      while (1)
      {
            flag = IORD_ALTERA_AVALON_PIO_DATA(SWITCHES_BASE);
            IOWR_ALTERA_AVALON_PIO_DATA(LEDS_BASE, flag &
0xff);
      }

}
```

Figure 26 Test switches and leds final code

To compile the new code from the `niosv-shell` run:

```
make -C software/app/debug
```

Then, you can execute it by running the *ELF_download* configuration. From the *Ashling RiscFree IDE for Intel FPGAs* select **Run > Run Configurations**, select *ELF_download* and click *run.*

Moving the switches you see the LEDs turning on and off.

To see the "Led and switch test" string on the console, you need to run the *juart-terminal*: **Run > External Tools > External Tools Configurations**, as described in Section 4.c (Apply External Tool Configuration and execute application).

In case the configuration *ELF_download* does not start, remove it: from **Run > Run Configurations**, select *ELF_download,* right click and select delete. Then, recreate the configuration as described in Section 4.b (Download the software ELF file).

6. Debugging

If the behavior of your program is not the expected one you can debug your code. Before debugging any system, you need to have at least one breakpoint to suspend the execution inside the program.

a. Open the *Ashling RiscFree IDE for Intel FPGAs*.
b. Open the Nios V processor software project (*hello.c*).
c. Set a breakpoint at the line

```
flag = IORD_ALTERA_AVALON_PIO_DATA(SWITCHES_BASE);
```

by double-clicking on the left-margin of the line.
d. Ensure that a blue circle have appeared beside the line.

To start the debugger:

a. In the *Run* menu tab, select *Debug Configurations*.
b. Select *ELF_download* to start the Debugger using the same settings used for the Run configuration and click *Debug*.

If the system proposes to switch to the debug perspective, select *switch.*
Then, the system starts the execution of your program and it stops on the line where you set the breakpoint, as shown in Figure 27. From the Run menu you can perform step-by-step debugging. You can see that on the right hand side of the *Ashling RiscFree IDE for Intel FPGAs* you can monitor variables content and check they take the expected value, as shown in Figure 28.

```
#include "system.h"
#include "sys/alt_stdio.h"
#include "altera_avalon_pio_regs.h"


int main ()
{
    int flag;

    alt_putstr("Led and switch test\n");

    while (1)
    {
        flag = IORD_ALTERA_AVALON_PIO_DATA(SWITCHES_BASE);
        IOWR_ALTERA_AVALON_PIO_DATA(LEDS_BASE, flag & 0xff);
    }

}
```

Figure 27 Debug your code

NOTE: In case the configuration *ELF_download* does not start, remove it: from **`Run > Debug Configurations`**, select *ELF_download,* right click and select delete. Then, recreate the configuration as described in Section 4.b (Download the software ELF file).

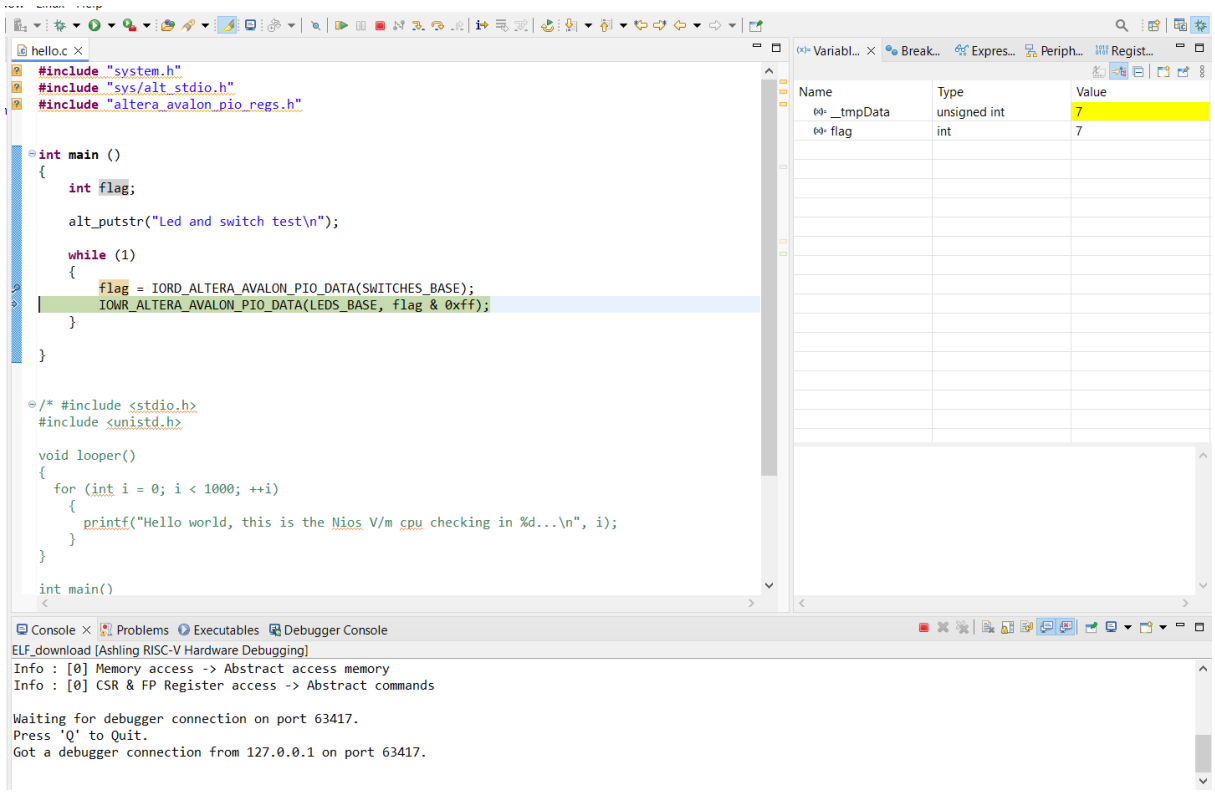


Figure 28 Test switches and leds debug

Part of this tutorial is derived from Intel-Altera document "*Introduction to the QSYS (Platform Designer) System Integration Tool*".

Part of this tutorial is derived from the "Nios EDF flow" material prepared by Prof. F. Gregoretti, DET, Politecnico di Torino.